Linear Regression for Classification

Advanced Machine Learning

Sam Ogden

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Learning objectives

- Give categories their own model output, via one-hot labels and softmax
- Define cross-entropy loss for that output
- Show its gradient looks just like linear regression's

Classification

The Big Question

Predicting *which*, not *how much*

What we have vs. what's different

What we already have

- A dataset, a model, a loss
- A way to train it — all from last lecture
- All of it assumed the answer is a single number

What's different now

- The answer is a **category**
  - `spam` or `not spam`
  - `cat` or `dog`
  - which digit
- No natural "how far off" between categories, the way there is between numbers

The Big Idea

Some questions don’t have a numeric answer — they have a category as the answer.

A concrete example we’ll carry through

We’ll sort pieces of fruit into **apple**, **banana**, or **orange**, from two features:

weight (g)	color score (0=green, 1=red)	fruit
150	0.9	apple
120	0.2	banana
180	0.7	orange

Why not just reuse linear regression directly?

Categories aren’t numbers

- We’re used to fitting a line to numbers
- Apple, banana, orange aren’t numbers — there’s no natural “less than” or “far apart” between them
- Coding them as 0, 1, 2 invents structure that isn’t really there

The Big Idea

Each category is binary — it is, or it isn't.

Let's just try it anyway

- Code the labels as integers: apple = 0, banana = 1, orange = 2
- Fit the usual linear model against those codes:

$$\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$$

- Predict a class by rounding $\hat{y}$ to the nearest code

Problem 1: we invented an ordering

- This encoding says banana (1) sits *between* apple (0) and orange (2)
- It says orange *minus* apple equals two *whole* bananas
- Relabel the fruits in a different order and the model's behavior changes

Problem 2: the output doesn't match the question

- $\hat{y}$ can be anything — `-1.3`, `4.7`, whatever the line gives
- Rounding to the nearest code is a patch, not a fix
- A `1.99` and a `1.01` both round to "banana," but they don't feel equally confident

What we actually need

- A way to write “which category” without implying order or distance → **one-hot encoding**
- A model output that’s bounded and behaves like a confidence → **softmax**

One-hot encoding

The Big Idea

Placeholder: give every category its own slot, and just mark which one is true.

A linear model, per class

The Big Idea

Placeholder: score every class the same dot-product way as before — just once per class instead of once total.

From scores to probabilities: softmax

The Big Idea

Placeholder: turn raw scores into something that behaves like a probability, without changing which class wins.

Measuring how wrong we are: cross-entropy

The Big Idea

Placeholder: penalize the model for how little probability it gave the correct answer.

Why not just use MSE here?

The Big Idea

Placeholder: the loss should punish confident wrongness harder than MSE does.

Differentiating the loss

The Big Idea

Placeholder: this scary-looking loss has a gradient just as simple as linear regression's.

Putting it together: training a classifier

The Big Idea

Placeholder: nothing about the training loop changed — only the loss and what "prediction" means.

Wrap